

Java Programming

A Practical Reference Guide

Core Language, Object-Oriented Programming, Collections, Generics,
Exception Handling, File I/O, and the Standard Library

Compiled Reference Edition

Table of Contents

1. Introduction to Java
2. Setting Up and Running Java Code
3. Variables, Data Types, and Operators
4. Strings and String Formatting
5. Arrays and the Collections Framework
6. Control Flow: Conditionals and Loops
7. Methods
8. Object-Oriented Programming
9. Interfaces and Abstract Classes
10. Exception Handling
11. File Input and Output
12. Generics
13. Useful Standard Library Classes
14. Build Tools and Best Practices
15. Quick Reference Tables

1. Introduction to Java

Java is a general-purpose, object-oriented programming language released by Sun Microsystems in 1995. Java source code is compiled to an intermediate form called bytecode, which runs on the Java Virtual Machine (JVM), letting the same compiled program run on any platform with a compatible JVM installed.

1.1 Key Characteristics

- **Platform independent:** The slogan "write once, run anywhere" reflects Java's compile-to-bytecode model executed by the JVM.
- **Statically typed:** Every variable's type is checked at compile time, catching many mistakes before the program runs.
- **Object-oriented:** Nearly everything in Java is expressed in terms of classes and objects, aside from a small set of primitive types.
- **Automatic memory management:** The JVM's garbage collector reclaims memory used by objects that are no longer reachable.
- **Large ecosystem:** A vast standard library plus build tools like Maven and Gradle give access to a huge range of third-party libraries.

1.2 Philosophy

Java emphasizes portability, readability, and robustness. Its strict type system, mandatory exception handling for checked exceptions, and automatic memory management are all designed to help large teams build and maintain reliable software over time.

2. Setting Up and Running Java Code

2.1 Installing the JDK

To write and run Java programs you need the Java Development Kit (JDK), which includes the compiler (javac) and the runtime (java). After installing, verify the version from a terminal:

```
$ java --version
java 21.0.2 2024-01-16 LTS
```

2.2 Compiling and Running a Program

Java source files end in `.java` and must contain a class whose name matches the file name. The compiler produces a `.class` bytecode file, which the JVM then executes.

```
// Hello.java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}

// Compile with: javac Hello.java
// Run with:     java Hello
// Output:      Hello, World!
```

2.3 The main Method

Every standalone Java application starts execution in a `public static void main(String[] args)` method. `static` means it belongs to the class itself rather than to an instance, and `args` holds any command-line arguments.

Note: Modern JDKs (11+) also support running a single source file directly with `java Hello.java`, without a separate compile step, which is handy for quick scripts.

3. Variables, Data Types, and Operators

3.1 Declaring Variables

Java variables are statically typed and must be declared with a type before use. Java 10 introduced `var` for local type inference.

```
int age = 30;
double price = 19.99;
char grade = 'A';
boolean isValid = true;
String name = "Alice";

// Local type inference (Java 10+)
var count = 10;      // inferred as int
var ratio = 3.14;    // inferred as double
```

3.2 Primitive Data Types

Type	Description	Size
int	Signed integer	4 bytes
short	Short signed integer	2 bytes
long	Long signed integer	8 bytes
float	Single-precision floating point	4 bytes
double	Double-precision floating point	8 bytes
char	Single 16-bit Unicode character	2 bytes
boolean	true or false	1 bit (JVM-dependent)
byte	Small signed integer	1 byte

3.3 Constants and Wrapper Classes

```
final double PI = 3.14159;    // final means the reference cannot change

// Wrapper classes box primitives as objects
Integer boxed = 42;           // autoboxing
int unboxed = boxed;          // auto-unboxing
```

3.4 Operators

- **Arithmetic:** `+` `-` `*` `/` `%`
- **Comparison:** `==` `!=` `<` `>` `<=` `>=`
- **Logical:** `&&` `||` `!`

- **Assignment:** = += -= *= /=
- **Increment/decrement:** ++ --
- **Ternary:** condition ? valueIfTrue : valueIfFalse

4. Strings and String Formatting

4.1 The String Class

Java strings are immutable objects of the `String` class; every operation that appears to modify a string actually returns a new one.

```
String first = "Hello";
String full = first + ", World!";
System.out.println(full.length());           // 13
System.out.println(full.substring(0, 5));    // "Hello"
System.out.println(full.toUpperCase());
boolean has = full.contains("World");        // true
```

4.2 StringBuilder

Because `String` is immutable, building up text in a loop with `+` is inefficient. `StringBuilder` provides a mutable buffer for that purpose.

```
StringBuilder sb = new StringBuilder();
sb.append("Hello");
sb.append(", ");
sb.append("World!");
String result = sb.toString();
```

4.3 String Formatting

```
String name = "Alice";
int age = 30;

// printf-style formatting
System.out.printf("%s is %d years old%n", name, age);

// text blocks (Java 15+)
String html = """
    <html>
        <body>Hello</body>
    </html>
    """;
```

5. Arrays and the Collections Framework

5.1 Arrays

Java arrays are fixed-size, and every element must share the same declared type. Array length is available via the `length` field.

```
int[] numbers = {1, 2, 3, 4, 5};
System.out.println(numbers[0]);    // 1
System.out.println(numbers.length); // 5

for (int n : numbers) {
    System.out.print(n + " ");
}
```

5.2 ArrayList

`ArrayList`, from `java.util`, is a resizable list backed by an array and is the most common general-purpose list implementation.

```
import java.util.ArrayList;
import java.util.List;

List<String> names = new ArrayList<>();
names.add("Alice");
names.add("Bob");
names.remove("Bob");
System.out.println(names.size());    // 1

for (String name : names) {
    System.out.println(name);
}
```

5.3 Other Collection Types

Interface / Class	Package	Use case
<code>ArrayList</code>	<code>java.util</code>	Resizable, ordered list
<code>LinkedList</code>	<code>java.util</code>	Doubly linked list / deque
<code>HashSet</code>	<code>java.util</code>	Unordered, unique elements
<code>TreeSet</code>	<code>java.util</code>	Sorted, unique elements
<code>HashMap</code>	<code>java.util</code>	Unordered key-value pairs
<code>TreeMap</code>	<code>java.util</code>	Sorted key-value pairs
<code>ArrayDeque</code>	<code>java.util</code>	Stack or queue operations

6. Control Flow: Conditionals and Loops

6.1 if / else if / else

```
int score = 85;
if (score >= 90) {
    System.out.println("A");
} else if (score >= 80) {
    System.out.println("B");
} else {
    System.out.println("C or below");
}
```

6.2 switch Statements and Expressions

```
int day = 3;

// Traditional switch statement
switch (day) {
    case 1:
        System.out.println("Monday");
        break;
    default:
        System.out.println("Another day");
}

// Modern switch expression (Java 14+)
String name = switch (day) {
    case 1 -> "Monday";
    case 2 -> "Tuesday";
    default -> "Another day";
};
```

6.3 Loops

```
// for loop
for (int i = 0; i < 5; i++) {
    System.out.print(i + " ");
}

// enhanced for loop (for-each)
int[] nums = {10, 20, 30};
for (int n : nums) {
    System.out.print(n + " ");
}

// while loop
int count = 0;
while (count < 3) {
    System.out.print(count);
    count++;
}

// do-while loop
```

```
int n = 0;
do {
    System.out.print(n);
    n++;
} while (n < 3);
```

Note: Use `break` to exit a loop early and `continue` to skip to the next iteration.

7. Methods

7.1 Defining Methods

A Java method declares a return type, a name, and a parameter list, and always belongs to a class.

```
public class MathUtils {  
    public static int add(int a, int b) {  
        return a + b;  
    }  
}  
  
int result = MathUtils.add(2, 3);    // 5
```

7.2 Method Overloading

Java allows multiple methods with the same name as long as their parameter lists differ in type or number, a feature called overloading.

```
int multiply(int a, int b) { return a * b; }  
double multiply(double a, double b) { return a * b; }  
  
multiply(2, 3);        // calls the int version  
multiply(2.5, 4.0);    // calls the double version
```

7.3 Varargs

```
static int sum(int... numbers) {  
    int total = 0;  
    for (int n : numbers) total += n;  
    return total;  
}  
  
sum(1, 2, 3);          // 6  
sum(1, 2, 3, 4, 5);    // 15
```

7.4 Pass-by-Value Semantics

Java always passes arguments by value. For primitives, a copy of the value is passed; for objects, a copy of the reference is passed, so the method can modify the object's fields but cannot reassign the caller's variable to a different object.

8. Object-Oriented Programming

8.1 Classes and Objects

```
public class BankAccount {
    private String owner;
    private double balance;

    public BankAccount(String owner, double balance) {
        this.owner = owner;
        this.balance = balance;
    }

    public void deposit(double amount) {
        balance += amount;
    }

    public double getBalance() {
        return balance;
    }
}

BankAccount acc = new BankAccount("Alice", 100.0);
acc.deposit(50.0);
System.out.println(acc.getBalance());    // 150.0
```

8.2 Access Modifiers

- **public:** accessible from any class.
- **protected:** accessible within the same package and by subclasses.
- **(package-private, no modifier):** accessible only within the same package.
- **private:** accessible only within the declaring class.

8.3 Inheritance

```
public class Animal {
    public void speak() {
        System.out.println("Some generic sound");
    }
}

public class Dog extends Animal {
    @Override
    public void speak() {
        System.out.println("Woof!");
    }
}

Animal a = new Dog();
a.speak();    // "Woof!" (runtime polymorphism)
```

8.4 equals(), hashCode(), and toString()

```
@Override
public String toString() {
    return "BankAccount{owner=" + owner + ", balance=" + balance + "}";
}

@Override
public boolean equals(Object o) {
    if (!(o instanceof BankAccount other)) return false;
    return owner.equals(other.owner) && balance == other.balance;
}
```

9. Interfaces and Abstract Classes

9.1 Interfaces

An interface declares a set of methods that implementing classes must provide, describing a contract without an implementation. A class can implement multiple interfaces.

```
public interface Shape {
    double area();
    double perimeter();
}

public class Circle implements Shape {
    private double radius;

    public Circle(double radius) { this.radius = radius; }

    @Override
    public double area() { return Math.PI * radius * radius; }

    @Override
    public double perimeter() { return 2 * Math.PI * radius; }
}
```

9.2 Abstract Classes

An abstract class can mix implemented methods with abstract ones that subclasses must fill in, and cannot itself be instantiated.

```
public abstract class Employee {
    protected String name;

    public Employee(String name) { this.name = name; }

    public abstract double calculatePay();

    public void printPaycheck() {
        System.out.println(name + ": $" + calculatePay());
    }
}
```

9.3 Default Methods

Since Java 8, interfaces may provide default implementations, allowing new methods to be added without breaking existing implementing classes.

```
public interface Greeter {
    default void greet() {
        System.out.println("Hello!");
    }
}
```

10. Exception Handling

10.1 try, catch, and finally

Java handles runtime errors with exceptions. Code that might fail is wrapped in a `try` block, errors are caught in `catch` blocks, and `finally` always runs regardless of whether an exception occurred.

```
public static double divide(double a, double b) {
    if (b == 0) {
        throw new IllegalArgumentException("Division by zero");
    }
    return a / b;
}

try {
    double result = divide(10, 0);
} catch (IllegalArgumentException e) {
    System.out.println("Error: " + e.getMessage());
} finally {
    System.out.println("Done attempting the division.");
}
```

10.2 Checked vs. Unchecked Exceptions

Checked exceptions (subclasses of `Exception` other than `RuntimeException`) must be declared or handled at compile time. Unchecked exceptions (subclasses of `RuntimeException`) do not require this.

Exception	Meaning
<code>NullPointerException</code>	Accessing a member on a null reference
<code>ArrayIndexOutOfBoundsException</code>	Invalid array index
<code>NumberFormatException</code>	Invalid string-to-number conversion
<code>IOException</code>	Checked exception for I/O failures
<code>IllegalArgumentException</code>	An argument does not have a valid value

10.3 try-with-resources

```
try (BufferedReader reader = new BufferedReader(new FileReader("f.txt"))) {
    System.out.println(reader.readLine());
} catch (IOException e) {
    System.out.println("Failed to read file: " + e.getMessage());
}
```

11. File Input and Output

11.1 Writing to a File

```
import java.io.FileWriter;
import java.io.IOException;

try (FileWriter writer = new FileWriter("data.txt")) {
    writer.write("Hello, file!\n");
    writer.write("42\n");
} catch (IOException e) {
    System.out.println("Write failed: " + e.getMessage());
}
```

11.2 Reading from a File

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

try (BufferedReader reader = new BufferedReader(new FileReader("data.txt"))) {
    String line;
    while ((line = reader.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    System.out.println("Read failed: " + e.getMessage());
}
```

11.3 The java.nio.file API

Since Java 7, the `java.nio.file` package offers a more modern, concise API for common file operations.

```
import java.nio.file.*;
import java.util.List;

List<String> lines = Files.readAllLines(Path.of("data.txt"));
Files.writeString(Path.of("out.txt"), "Hello, NIO!");
```

12. Generics

12.1 Generic Classes

Generics let a class or method work with any type while keeping compile-time type safety, instead of relying on raw `Object` types and casts.

```
public class Box<T> {  
    private T value;  
  
    public Box(T value) { this.value = value; }  
  
    public T get() { return value; }  
}
```

```
Box<Integer> intBox = new Box<>(42);  
Box<String> strBox = new Box<>("hello");
```

12.2 Generic Methods

```
public static <T> T firstElement(List<T> list) {  
    return list.get(0);  
}
```

```
String first = firstElement(List.of("a", "b", "c")); // "a"
```

12.3 Bounded Type Parameters

```
public static <T extends Comparable<T>> T max(T a, T b) {  
    return a.compareTo(b) >= 0 ? a : b;  
}
```

```
max(3, 7); // 7  
max("apple", "banana"); // "banana"
```

13. Useful Standard Library Classes

13.1 The Streams API

Introduced in Java 8, the Streams API supports declarative, functional-style operations over collections, such as filtering, mapping, and reducing.

```
import java.util.List;
import java.util.stream.Collectors;

List<Integer> nums = List.of(1, 2, 3, 4, 5);
List<Integer> evenSquares = nums.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * n)
    .collect(Collectors.toList());
// [4, 16]

int total = nums.stream().mapToInt(Integer::intValue).sum(); // 15
```

13.2 Optional

```
import java.util.Optional;

Optional<String> maybeName = Optional.ofNullable(getName());
String name = maybeName.orElse("Unknown");
maybeName.ifPresent(n -> System.out.println("Found: " + n));
```

13.3 Math, Random, and LocalDate

```
import java.util.Random;
import java.time.LocalDate;

double root = Math.sqrt(16);           // 4.0

Random random = new Random();
int n = random.nextInt(10);            // 0-9

LocalDate today = LocalDate.now();
System.out.println(today.plusDays(7));
```

13.4 Wrapper and Utility Classes

```
Integer.parseInt("42");                // String -> int
String.valueOf(42);                    // int -> String
Collections.sort(myList);
Arrays.asList(1, 2, 3);
Objects.requireNonNull(value, "value must not be null");
```

14. Build Tools and Best Practices

Real Java projects rarely compile files by hand; build tools manage dependencies, compilation, testing, and packaging.

14.1 Maven

```
<!-- pom.xml -->
<project>
  <groupId>com.example</groupId>
  <artifactId>my-app</artifactId>
  <version>1.0.0</version>
</project>

# Build:
$ mvn compile
$ mvn test
$ mvn package
```

14.2 Gradle

```
// build.gradle
plugins { id 'java' }
group = 'com.example'
version = '1.0.0'

# Build:
$ gradle build
$ gradle test
```

14.3 Style Guidelines

- Use PascalCase for class names and camelCase for methods and variables.
- Use UPPER_CASE for constants declared `static final`.
- Prefer interfaces (e.g. `List`) as variable types over concrete implementations (e.g. `ArrayList`).
- Keep classes focused on a single responsibility.
- Always close resources, preferably with `try-with-resources`.
- Write Javadoc comments for public classes and methods.

15. Quick Reference Tables

15.1 Common java.lang and java.util Classes

Class	Purpose
String	Immutable text
StringBuilder	Mutable text buffer
ArrayList	Resizable list
HashMap	Key-value store
Math	Static math functions
Scanner	Read console/text input
Objects	Null-safe utility methods
Optional	Container that may or may not hold a value

15.2 Common Compiler/Runtime Commands

```
javac Hello.java      # compile a single file
java Hello            # run a compiled class
java Hello.java       # compile and run in one step (Java 11+)
javac -d out src/*.java # compile into an output directory
jar cf app.jar -C out . # package classes into a JAR
```

15.3 Common Stream Patterns

```
list.stream().filter(x -> x > 0).count();
list.stream().sorted().collect(Collectors.toList());
list.stream().anyMatch(x -> x.equals("target"));
list.stream().map(String::toUpperCase).forEach(System.out::println);
list.stream().reduce(0, Integer::sum);
```

This guide is a condensed, original reference summarizing core Java concepts and is not a reproduction of the official Java documentation. For complete and authoritative details, consult the official docs at docs.oracle.com/en/java.